

설비 고장을 예측하는 AI 데이터 분석

이상탐지 모델 적용

IsolationForest 이상탐지

C O N T E N T S

목차

01 **IsolationForest 원리**
고립의 원리 · 핵심 파라미터

02 **모델 적용과 결과 해석**
predict와 점수 · 심각도 순위

03 **파라미터와 활용**
새 데이터 적용 · Z-score 비교

01

IsolationForest 원리

고립의 원리 · 핵심 파라미터

Z-score만으로 부족한 진짜 이유

특징을 하나씩만 보는 구조라 다변량 이상은 구조상 놓침

01 · 시야의 좁음

Z-score는 특징을 **하나씩만** 봐서 관계가 무너지는 이상을 놓침

02 · 관계의 어긋남

정상 기계는 특징들이 함께 움직이는데, 고장은 그 관계가 어긋남

03 · 구조적 한계

임계값을 아무리 조정해도 구조상 다변량 이상은 못 잡음

여러 특징을 통째로 보는 도구

IsolationForest는 여러 특징을 한꺼번에 봄

01 · 한 쌍으로 묶기

rms와 거칠기를 한 쌍으로
묶어 **무리에서 떨어졌는지**
판단

02 · 관계 이상 포착

특징을 따로 안 보니
Z-score가 놓친
관계의 이상을 잡음

03 · 실제 성능

우리 데이터의
다변량 이상
16개 중 절반을 잡아냄

트리로 이상을 찾는 발상

예-아니오 질문으로 데이터를 둘로 계속 나누는 트리

01 · 트리 구조

트리는 예-아니오 질문을 던져 데이터를 둘로 계속 나누는 구조

02 · 반복의 끝

질문을 거듭하면 결국 각 데이터가 혼자 남는 칸에 도달

03 · 핵심 단서

혼자가 되기까지 걸린 질문 수가 정상.이상의 단서

정상은 늦게, 이상은 빨리 혼자가 됨

혼자가 되기까지 걸린 질문 수로 이상을 가림

01 · 정상

비슷한 친구가 많아
한참 쪼개야
겨우 혼자가 됨

02 · 이상

무리에서 동떨어져
몇 번만에 뚝
혼자가 됨

03 · 판정 규칙

적게 걸리면 이상 의심
많이 걸리면 정상

스무고개 비유로 이해하는 고립

특이할수록 적은 질문으로 콕 집힘

01 · 사과

비슷한 과일과 헛갈려
질문이 많이 필요

무리 속에 속한 정상

02 · 에펠 탑

사람이 만든 건가
아주 높은가
두세 질문이면 끝

무리에서 떨어진 이상

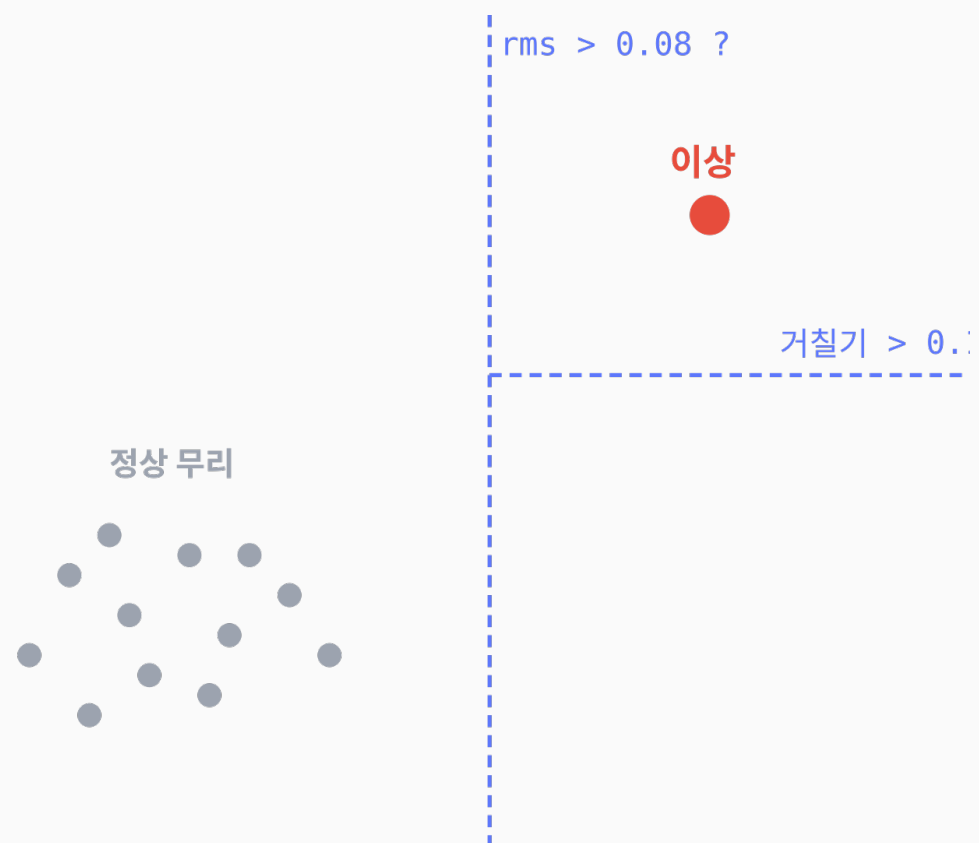
03 · 핵심

이상은 에펠탑처럼
적은 질문으로
고립 (Isolation)

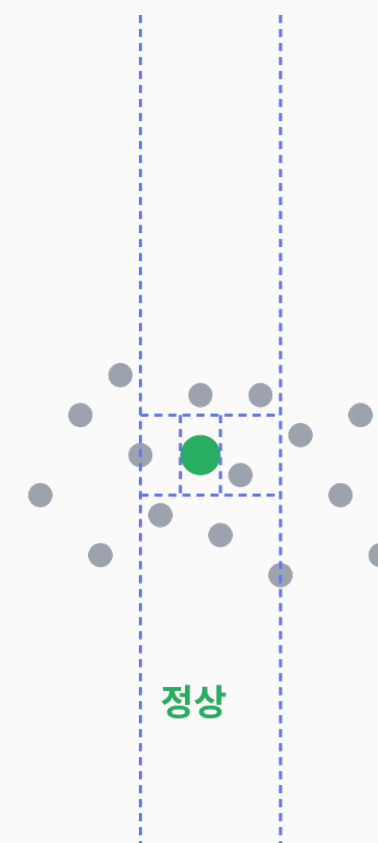
고립 — 한눈에 보기

이상은 2~3번 분할로, 정상은 여러 번 분할해야 혼자가 됨

이상 · 적은 질문 (2~3번)



정상 · 많은 질문 (층층히 여러 번)



IsolationForest의 핵심 — 고립

정상을 그리지 않고 곧장 이상을 떼어내는 발상

01 · 분할로 고립

무작위 질문으로 데이터를 쪼개 **각 점을 고립**시킴

02 · 적은 분할 = 이상

적은 분할로 고립될수록 이상으로 판단

03 · Z-score와의 차이

정상을 그리지 않고 곧장 이상을 떼어내는 방식

이상이 빨리 고립되는 이유

무리에서 멀수록 적은 분할로 고립됨

01 · 구석의 이상

구석에 외따로 떨어진
이상 — 선 한두 번에
혼자 영역 생김

02 · 무리 속 정상

빽빽한 무리 속 정상 —
친구를 떼어내려면
쭈뼌 여러 번

03 · 효과

거리를 직접 안 재고
분할 횟수만 세도
거리감을 얻음

거리 대신 분할 횟수

분할 횟수가 거리 계산을 대신함

01 · 분할의 의미

무리에서 멀수록 적은 분할, 한가운데일수록 많은 분할

02 · 거리 대신

이 단순한 관계가 **거리 계산을 대신함**

03 · 설비 데이터 적합

특징이 많아져도 가벼워 설비 데이터에 잘 맞음

무작위 분할로 나무를 만듦

나무 한 그루가 자라는 과정

01 · 특징 무작위 선택

여러 특징 중 하나를
무작위로 고름
(예 rms)

02 · 분할 지점 무작위

값 범위에서
분할 지점을 무작위로
(예 0.072)

03 · 반복

둘로 가르고, 각 무리에
같은 일을
혼자 남을 때까지 반복

혼자가 되기까지 거친 분할의 깊이가 그 데이터의 고립 깊이

무작위라서 빠르고, 평균으로 다스림

한 그룹은 운에 좌우 · 여러 그룹 평균으로 흔들림 잡기

01 · 속도의 비결

특징과 분할점을 똑똑히 계산 안 하고 무작위로 정해 빠름

02 · 무작위라도 OK

이상은 워낙 동떨어져 아무렇게 잘라도 금방 고립됨

03 · 평균으로 안정화

한 그룹은 운에 좌우 — 여러 그룹 평균으로 흔들림을 잡음

아무렇게 잘라도 이상은 동떨어져 있어 금방 고립됨 · 평균이 우연을 다스림

고립 깊이가 이상 점수가 됨

얼마나 이상한가라는 정도까지 알려주는 점수

01 · 깊이의 의미

깊이가 **얕으면 이상**, 깊으면 정상

02 · 0~1 점수 환산

이 깊이를 0~1 사이 이상 점수로 환산 (얕을수록 1에 가까움)

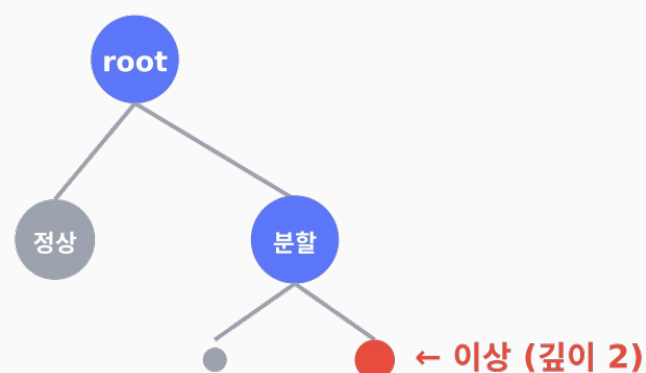
03 · 우선순위

정도까지 알면 가장 이상한 순서로 우선순위 매기기 가능

고립 깊이와 점수 — 한눈에 보기

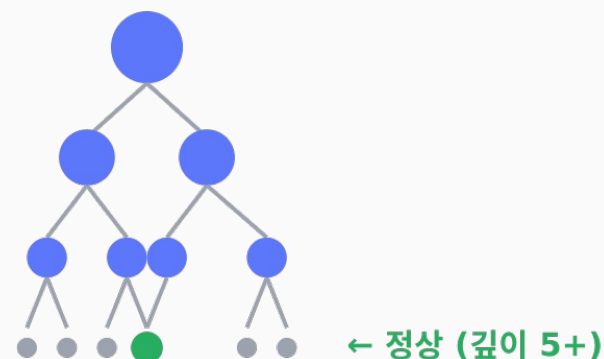
얕은 트리는 이상(점수 높음), 깊은 트리는 정상(점수 낮음)

얕은 트리 · 2회 분할로 고립 · 이상 점수 높음



고립 깊이 → 0~1 점수
점수 ≈ 0.8 (이상)

깊은 트리 · 여러 분할 끝에 고립 · 이상 점수 낮음



고립 깊이 → 0~1 점수
점수 ≈ 0.4 (정상)

핵심 한 줄

무작위로 쪼개고
깊이를 세고
점수로 바꿈

거리도 안 재고,
정상을 모형으로
그리지도 않았는데
정도까지 매겨 줌

여러 나무의 평균 — Forest

한 그루는 못 믿어도 여러 그루의 평균은 믿을 만함

01 · 한 그루의 한계

나무 한 그루는 무작위라 운에 흔들림

02 · 평균으로 안정

수십~수백 그루를 만들어 고립 깊이의 **평균**을 냄

03 · 심사위원 비유

여러 심사위원의 평균 점수가 공정하듯, 우연이 묻혀 안정적

contamination 파라미터

z-score 임계값과 비슷한 역할, 가장 중요한 설정값

01 · 비율 다이얼

contamination은 전체 중 이상 비율을 알려주는 다이얼

02 · 값의 의미

0.1이면 점수 높은 상위 약 10%를 이상으로 표시

03 · 트레이드오프

임계값과 같은 트레이드오프 — 키우면 많이, 줄이면 적게

contamination에 따른 탐지 개수

실습 데이터 220개 (실제 이상 40개, 약 18%)

contamination = 0.05

약 11개

확실한 것만 표시

많이 놓침

contamination = 0.18

약 40개

실제 비율에 맞춤

균형

contamination = 0.25

약 55개

다 잡지만 거짓 경보 높

민감

현장에선 정확한 이상 비율을 모르니 작게 시작해 결과를 보며 조정

n_estimators와 random_state

안정성과 재현성을 담당하는 두 파라미터

n_estimators

나무 개수, 많을수록 안정적

키우면 더 안정적이지만 느려짐
어느 정도부터는 차이가 작음

기본값 **100** 이면 대부분 충분

역할 — 안정성

random_state

무작위 결과 재현

돌릴 때마다 결과가 조금씩 달라짐
같은 숫자를 넣으면 항상 같은 결과

실습은 **42** 로 고정

역할 — 재현성

마법 주사위 — 재현되는 무작위

random_state를 고정하면 무작위가 똑같이 반복됨

01 · 진짜 주사위

굴릴 때마다
다른 눈

일반 무작위

02 · 마법 주사위

항상 같은 순서로
눈이 나옴

재현되는 무작위

03 · 실습 약속

random_state=42

검증과 비교가 쉬워짐

학습 데이터에 이상이 섞여도 되나

IsolationForest는 오염에 너그러움

01 · 견딤

학습 데이터에 이상이 조금 섞여도 견딤

02 · 이유

평균·표준편차에 의존 안 하고, 이상은 어차피 소수라 빨리 고립

03 · 현장 이점

그래서 정상만 깔끔히 골라낼 부담이 줄어들음

너그러움의 한계

소수의 동떨어진 이상이라는 전제 안에서만 성립

01 · 조금 섞임

이상이 조금 섞인 정도
→ 견딜

02 · 절반 가까이

이상이 절반 가까이
→ 전제 깨짐
제대로 작동 못 함

03 · 적합 상황

보통 설비처럼
정상이 대부분일 때
현장에서 쓰기 좋음

StandardScaler — 특징 범위 맞추기

트리 기반엔 영향이 작은 전처리

01 · 기능

특징마다 다른 값 범위를 평균 0·표준편차 1로 맞추는 도구

02 · 계산식

사실 z-score와 똑같은 계산 $(\text{값} - \text{평균}) / \text{표준편차}$

03 · 모델별 효과

거리 기반 방법엔 거의 필수, 트리 기반엔 영향이 작음

트리는 한 특징 안에서 크다 작다만 묻는 분할이라 스케일에 휘둘리지 않음

전처리는 효과를 확인하고 씬

좋다고 알려진 전처리도 모델에 따라 효과가 다름

01 · 트리 기반의 특성

트리 기반 IsolationForest
스케일링 효과 작음

02 · 실습에서 확인

실습에서 적용 전후를
비교해 효과를
눈으로 확인

03 · 원칙

무작정 넣지 말고
우리 데이터·모델에서
확인하고 판단

scikit-learn 사용 흐름 — 3단계

모든 scikit-learn 모델은 만들기 → fit → predict 공통 흐름

STEP 1

만들기

설정만 갖춘 빈 모델

```
IsolationForest(...)
```



STEP 2

fit · 학습

숲을 실제로 짓는 단계, 보통 한 번

```
iso.fit(X)
```



STEP 3

predict · 예측

새 데이터마다 다시 훈련 없이 반복

```
iso.predict(X)
```

fit과 predict — 코드 3줄

길게 배운 고립과 숲의 원리가 단 세 줄로 압축됨

ISOLATIONFOREST.PY

```
from sklearn.ensemble import IsolationForest

X = df[["rms", "spectral_centroid", "zero_crossing_rate"]]

iso = IsolationForest(contamination=0.18, random_state=42)
iso.fit(X)

pred = iso.predict(X)
# pred → -1 (이상) 또는 1 (정상)
```

특징 묶기 → 모델 만들기 → fit → predict, predict 결과는 -1과 1

02

모델 적용과 결과 해석

predict와 점수 · 심각도 순위

실습 — IsolationForest 첫 적용

만들기-fit-predict 흐름으로 우리 데이터에 처음 적용

GOAL

만들기-fit-predict 흐름으로 우리 데이터에 처음 적용

STEPS

Step 1 `sklearn.ensemble`에서 `IsolationForest`를 import

Step 2 특징 3개만 골라 `X`로 만들기 (label은 넣지 않음)

Step 3 `IsolationForest(contamination=0.18, random_state=42)`로 모델 만들기

실습 — 학습하고 예측하기

fit으로 학습, predict로 예측해 결과값 확인

GOAL

학습된 모델을 만들고 예측 결과로 -1, 1의 의미 확인

STEPS

Step 4 `iso.fit(X)`로 학습시키기

Step 5 `pred = iso.predict(X)`로 예측하고 앞부분 출력

Step 6 `set(pred)`로 어떤 값이 들어 있는지 확인 (예상 -1과 1)

CHECKPOINT

predict 결과에 -1과 1 두 종류만 들어 있는가

길게 배운 고립과 숲의 원리가 단 세 줄로 실행됨

predict 결과 -1과 1의 의미

초보자가 가장 많이 헷갈리는 지점, scikit-learn의 약속

01 · 약속

predict는 이상을 -1, 정상을 1로 돌려줌

02 · 우리의 통념과 반대

우리가 익숙한 이상=1, 정상=0과 정반대

03 · 외우는 법

무리 바깥(outlier)이 -1, 무리 안(inlier)이 1로 외우기

우리 형식으로 변환하기

predict 결과를 익숙한 형식으로 바꿔 쓰기

01 · 변환 규칙

-1 → 이상(1)

1 → 정상(0)

으로 변환

02 · 조건식

(pred == -1)

이상인 곳만 참(True)

03 · 효과

label과 같은 기준으로
비교·채점 가능

이 약속을 놓치면 결과 전체가 뒤집힌다는 점 반드시 기억

실습 — 이상 개수 세어보기

이상(-1) 개수를 세고 정답과 비교해 가늠

GOAL

이상(-1) 개수를 세고 정답과 비교해 가늠

STEPS

Step 1 `pred`가 -1이면 이상(1), 1이면 정상(0)으로 `pred_anom` 컬럼 만들기

Step 2 `pred_anom`의 합으로 이상 개수 세기 (약 **40개**)

tip · contamination 0.18 → 약 40개. 앞 실습의 `pred`를 이어서 씀

실습 — 교차표로 비교

정답 label과 교차표로 비교해 잡힌 이상을 가늠

GOAL

정답 label과 교차표로 비교해 잡힌 이상을 가늠

STEPS

Step 3 `pd.crosstab`으로 label과 pred_anom의 교차표 만들기

Step 4 실제 이상인데 모델도 이상으로 잡은 수 확인 (약 **27개**)

Step 5 z-score가 잡았던 23개, 놓친 다변량 이상과 비교

CHECKPOINT

실제 이상 40개 중 약 27개를 잡았는가

decision_function과 score_samples

얼마나 이상한가라는 정도를 알려주는 점수

01 · 점수의 방향

이상의 정도를 점수로 — 값이 작을수록 이상

02 · 개념과의 부호 차이

개념상 점수(1에 가까울수록 이상)와 부호 방향이 반대

03 · 외우는 법

작을수록 이상, 클수록 정상으로 외우는 게 가장 안전

점수와 predict는 한 뿌리

점수를 보면 predict 판정의 이유까지 보임

01 · 내부 동작

predict는 내부적으로
이 점수를 먼저 구함

02 · 경계 결정

contamination이 정한
경계로 -1과 1을
나눔

03 · 해석 가능성

점수를 직접 보면
왜 그렇게 판정했는지
이해됨

이상 점수로 심각도 순위 매기기

점수가 작은 순서로 정렬하면 점검 우선순위가 됨

01 · 점수 매기기

모든 데이터에
이상 점수를 매김

02 · 정렬

작은 (가장 이상한)
순서로 정렬 →
맨 위가 1순위

03 · 유연한 활용

인력 사정에 맞춰
상위 3개·10개로
조절

점수는 정도일 뿐, 심각도가 아님

실제 위험도는 정비 지식과 함께 판단

01 · 점수의 의미

점수는 정상 무리에서 얼마나 떨어졌나를 나타냄

02 · 한계

많이 떨어진 것이 곧 위험한 고장이라는 보장은 없음

03 · 활용 원칙

순위는 참고로 쓰되, **특징값**을 함께 보여 줘야 단서가 됨

실습 — 이상 점수 정렬해 Top N

score_samples로 점수를 구해 가장 이상한 상위 N개 뽑기

GOAL

score_samples로 점수 구해 가장 이상한 상위 N개 뽑기

STEPS

Step 1 iso.score_samples(X)로 점수를 구해 score 컬럼 추가

Step 2 score가 작은 순서(오름차순)로 정렬

실습 — Top 10 해석하기

상위 10개를 뽑아 특징값으로 왜 이상한지 설명

GOAL

상위 10개를 뽑아 특징값으로 왜 이상한지 설명

STEPS

Step 3 상위 10개의 특징값·점수·label을 함께 보기

Step 4 상위 10개 중 실제 이상(label=1)이 몇 개인지 세기

Step 5 세기가 큰가, 톤·거칠기 조합이 특이한가로 이유 설명

CHECKPOINT

상위 10개 대부분이 실제 이상이고, 다변량 이상도 섞여 있는가

03

파라미터와 활용

새 데이터 적용 · Z-score 비교

contamination을 바꾸면 결과가 변함

비율을 바꿔도 점수의 순위 자체는 변하지 않음

01 · 다이얼

contamination은 이상으로 표시할 **비율을 정하는 다이얼**

02 · 트레이드오프

키우면 많이, 줄이면 적게 — 임계값과 같은 트레이드오프

03 · 순위는 유지

비율을 바꿔도 이상 점수의 순위 자체는 변하지 않음

contamination별 진짜 이상 비율

잡힌 것 중 진짜 이상이 얼마나 되는지가 핵심

contamination	잡힌 개수	진짜 이상	특성
0.05	약 11개	약 10개 진짜	정확하지만 많이 놓침
0.18	약 40개	약 27개 진짜	균형
0.25	55개	약 35개 진짜 / 20개 오탐	거짓 경보 높

적게 잡으면 정확하지만 놓치고, 많이 잡으면 다 잡지만 거짓 경보가 높 — 시소 관계

실습 — contamination 값 실험

여러 값으로 돌려 잡힌 개수와 진짜 이상 비율 변화 관찰

CASE A · 보수적 0.05

확실한 것만, 무엇을 놓치나

CASE B · 민감 0.25

많이 잡으면 거짓 경보가 얼마나

STEPS

Step 1 contamination 후보값 목록 만들기 (0.05, 0.10, 0.18, 0.25)

Step 2 각 값마다 fit-predict해 이상 개수 세기

값을 바꿔가며 여러 번 실행 · random_state는 42로 고정 · 약 11, 22, 40, 55개

실습 — 트레이드오프 표로 정리

잡힌 개수와 진짜 이상 수를 표로 정리해 트레이드오프 확인

STEPS

Step 3 각 값마다 잡힌 것 중 진짜 이상(label=1) 수도 세기

Step 4 contamination · 잡힌 개수 · 진짜 이상 수를 표로 정리

Step 5 비율을 키울 때 무엇이 늘고 주는지 거짓 경보·놓침으로 설명

CHECKPOINT

실제 비율(약 18%)에 맞춘 0.18이 균형이 좋은 편인가

KEY POINT

현장에선 정답 비율을 모름 — 추정해 조정하는 감각이 핵심

실습 — StandardScaler 적용 전

원본 특징으로 먼저 돌려 기준 결과를 얻기

GOAL

원본 특징으로 먼저 돌려 기준 결과를 얻기

STEPS

Step 1 원본 특징 **X**로 IsolationForest를 fit-predict

Step 2 진짜 이상(**label=1**)을 몇 개 잡았는지 세기 (약 **27개**)

이 결과가 비교 기준

실습 — 스케일링 후 비교

스케일링한 결과와 비교해 트리 기반의 특성 확인

STEPS

Step 3 `StandardScaler`로 `x`를 변환해 `X_scaled` 만들기

Step 4 `X_scaled`로 같은 설정 모델을 fit-predict, 진짜 이상 수 세기

Step 5 두 결과를 표로 비교하고 거의 같은 이유를 원리로 설명

CHECKPOINT

스케일링 전과 후 진짜 이상 탐지 수가 거의 같은가 (약 27개)

KEY POINT

트리 기반은 거리 대신 특징별 분할 — 크기 차이에 휘둘리지 않음

학습된 모델을 새 데이터에 적용

여기에 fit과 predict를 나눠 둔 진짜 이유

01 · 실제 운영 패턴

실제 운영은 한 번 학습(fit), 여러 번 예측(predict)

02 · 반복 사용

새 측정값이 들어올 때마다 학습된 모델로 predict만 반복

03 · 효과

빠르고, 같은 기준으로 보니 일관성이 보장됨

새 데이터는 형식을 맞춰 넣음

새 데이터도 학습 때와 똑같은 형식으로

01 · 특징 순서

학습 때와 같은
특징을 같은 순서로
넣기

02 · 전처리

학습 때 스케일링을
썼다면 새 데이터에도
같은 기준으로

03 · 흔한 실수

새 데이터로
다시 학습하면
기준이 흐트러짐

실습 — 새 데이터에 모델 적용

학습한 모델을 label 없는 새 데이터에 적용 (현장 상황)

GOAL

학습한 모델을 label 없는 새 데이터에 적용 (현장 상황)

STEPS

Step 1 `mimii_features.csv`의 특징으로 모델을 한 번 학습(fit)

Step 2 `mimii_new.csv`를 불러와 같은 3개 특징을 같은 순서로 `x_new` 만들기

label 없는 새 데이터 40행을 같은 특징·순서로 준비 (`mimii_new.csv`엔 label 없음)

실습 — 새 데이터 이상 가려내기

predict와 점수로 새 데이터의 이상을 가려내고 순위 매기기

STEPS

Step 3 학습된 모델로 `iso.predict(X_new)` 실행

Step 4 새 데이터에서 이상으로 잡힌 개수 세기 (약 **10개**)

Step 5 `iso.score_samples(X_new)`로 가장 이상한 순서로 정렬

CHECKPOINT

label이 없는데도 모델이 이상을 가려내는가 (비지도 핵심)

KEY POINT

한 번 학습 여러 번 예측 구조와 형식 맞추기 원칙을 직접 체험

Z-score와 IsolationForest 비교

무엇이 더 좋다가 아니라 각각 언제 쓰면 좋은지가 핵심

항목	Z-score	IsolationForest
보는 방식	특징 하나씩	여러 특징 동시에
다변량 이상	농침	일부 잡음
이상 섞인 학습	기준선 오염에 약함	어느 정도 견딤
적합 상황	빠른 1차 점검	관계형 이상 탐지

직관성은 z-score, 설정값 부담은 IsolationForest가 더 큼

보완 관계와 교차 확인

둘은 우열이 아니라 보완 — 겹쳐 보면 신뢰도가 오름

01 · 1차 점검

빠른 **Z-score**로
먼저 훑어
명백한 이상을 거름

02 · 관계 이상

조합이 의심되면
IsolationForest로
관계 이상까지

03 · 판단 규칙

둘 다 이상이면
강한 의심,
한쪽만이면
추가 확인 대상

실습 — 두 방법 결과 만들기

Z-score와 IsolationForest 이상 컬럼을 한 데이터에 모으기

GOAL

Z-score와 IsolationForest 이상 컬럼을 한 데이터에 모으기

STEPS

Step 1 모듈 1 방식으로 Z-score(임계값 3) 기반 `z_anom` 컬럼 만들기

Step 2 이번 모듈의 IsolationForest 기반 `if_anom` 컬럼 만들기

두 결과 컬럼을 한 데이터에 모아 교차 비교

실습 — 교차 확인하고 해석

두 결과를 교차해 일치·불일치 이상을 구분

STEPS

Step 3 `pd.crosstab`으로 둘 다 이상/한쪽만/둘 다 정상 개수 보기

Step 4 IsolationForest만 잡은 것이 다변량 이상인지 확인

Step 5 강한 의심과 추가 확인 대상을 어떻게 다르게 다룰지 이야기

CHECKPOINT

IsolationForest만 잡은 것이 세기 보통, 톤·거칠기 높은 이상인가

KEY POINT

경쟁이 아니라 보완으로 쓰는 감각 — 둘 다 이상이면 강한 의심

오탐과 미탐 — 모델이 틀리는 경우

완벽하진 않다는 사실을 정직하게 짚기

01 · 두 종류의 오류

오탐 — 정상인데 이상으로, **미탐** — 이상인데 정상으로

02 · 트레이드오프

같은 트레이드오프 — 많이 잡으면 오탐 늘고, 적게 잡으면 미탐 늘

03 · 회색 지대

정상과 이상이 겹치는 **회색 지대**는 어떤 도구도 헛갈림

모델은 판정이 아니라 점검 후보

완벽하지 않다는 걸 아는 게 도구를 가장 잘 쓰는 길

01 · 결과의 위치

최종 판정이 아니라
점검할 후보를
추려 줌

02 · 사람이 확인

모델이 이상이라 한 것을
정비사가 확인해
진짜인지 가림

03 · 좋은 분석가

모델이 어디서
틀리는지도
함께 설명

IsolationForest를 언제 쓰면 좋은가

도구를 아는 것과 언제 쓸지 아는 건 다른 능력

01 · 다변량 이상

여러 특징의 **관계**가 중요할 때 (다변량 이상)

02 · 오염 견딤

정상만 깔끔히 골라내기 어려울 때 (오염 견딤)

03 · 빠른 처리

데이터가 크고 특징이 많아 빠른 처리가 필요할 때

단순한 문제엔 단순한 도구

도구의 정교함이 항상 정답은 아님

01 · 단순한 문제

특징이 한두 개고
단순하면
빠른 Z-score로 충분

02 · 설명의 쉬움

누구나 이해하게
설명할 땐 Z-score의
직관성이 유리

03 · 최종 원칙

도구는
문제를 위해 존재,
문제에 맞는
도구를 고름